

Tareas supervisadas usando ELM

 UNIVERSIDAD
SIMÓN BOLÍVAR

BOGOTÁ Y COLOMBIA | VENEZUELA

CIENCIAS BÁSICAS
Y BIOMÉDICAS



Tareas supervisadas usando ELM.

- Este bloque de código:

- a) Permite elegir entre Regresión y Clasificación
- b) Carga, en memoria, los datos y **Etq** de **ENT**
- c) Carga, en memoria, los datos y **Etq** de **PRU**
- d) Identifica el número de ejemplos de **ENT** y **PRU**
- e) Establece el número de neuronas de entrada

```
##### Macro definition
REGRESSION=0;
CLASSIFIER=1;

##### Load training dataset
train_data=load(TrainingData_File);
T=train_data(:,1)';
P=train_data(:,2:size(train_data,2))';
clear train_data;

##### Load testing dataset
test_data=load(TestingData_File);
TV.T=test_data(:,1)';
TV.P=test_data(:,2:size(test_data,2))';
clear test_data;

NumberOfTrainingData=size(P,2);
NumberOfTestingData=size(TV.P,2);
NumberOfInputNeurons=size(P,1);
```

Tareas supervisadas usando ELM.

- Este bloque de código:

a) Concatena y ordena las **Etq** de **ENT** y **PRU**

b) Indica que las clases son iguales al número de **Etq**

c) Asigna el número de clases a las neuronas de salida

```
if Elm_Type==REGRESSION
    %%%%%%%%%%% Preprocessing the data of classificatio
    sorted_target=sort(cat(2,T,TV.T),2);
    label=zeros(1,1);
    label(1,1)=sorted_target(1,1);
    j=1;
    for i = 2:(NumberofTrainingData+NumberofTestingData)
        if sorted_target(1,i) ~= label(1,j)
            j=j+1;
            label(1,j) = sorted_target(1,i);
        end
    end
    number_class=j;
    NumberofOutputNeurons=number_class;
```

Tareas supervisadas usando ELM.

- Este bloque de código:

- a) Pre-procesa las **Etq** de **ENT**
- b) Compara las **Etq** ordenadas con las **Etq** de **ENT**
- c) Cambia las **Etq** originales de **ENT** por **Etq** genéricas

```
%%%%%%%% Processing the targets of training
temp_T=zeros(NumberOfOutputNeurons, NumberOfTrainingData);
for i = 1:NumberofTrainingData
    for j = 1:number_class
        if label(i,j) == T(i,i)
            break;
        end
    end
    temp_T(j,i)=1;
end
T=temp_T*2-1;
```

Tareas supervisadas usando ELM.

- Este bloque de código:

a) Pre-procesa las **Etq** de **PRU**

b) Compara las **Etq** ordenadas con las **Etq** de **PRU**

c) Cambia las **Etq** originales de **PRU** por **Etq** genéricas

```
%%%%%%%%%% Processing the targets of testing
temp_TV_T=zeros(NumberOfOutputNeurons, NumberOfTestingData);
for i = 1:NumberOfTestingData
    for j = 1:number_class
        if label(i,j) == TV.T(i,i)
            break;
        end
    end
    temp_TV_T(j,i)=i;
end
TV.T=temp_TV_T*1;
```

end



6:29 / 14:19



Tareas supervisadas usando ELM.

```
%%%%%%%%%% Random generate input weights InputWeight (w_i) and b
InputWeight=rand(NumberOfHiddenNeurons,NumberOfInputNeurons)*2-1;
BiasofHiddenNeurons=rand(NumberOfHiddenNeurons,1);
tempH=InputWeight*P;
clear P;
ind=ones(1,NumberOfTrainingData);
BiasMatrix=BiasofHiddenNeurons(:,ind);
tempH=tempH+BiasMatrix;
```

- En el **ENT**, este bloque de código:

- a) Genera los pesos de entrada al azar
- b) Genera los sesgos de entrada al azar
- c) Combina linealmente pesos y sesgos

Tareas supervisadas usando ELM.

```
switch lower(ActivationFunction)
case {'sig', 'sigmoid'}
    %%%%%%%%% Sigmoid
    H = 1 ./ (1 + exp(-tempH));
case {'sin', 'sine'}
    %%%%%%%%% Sine
    H = sin(tempH);
case {'hardlim'}
    %%%%%%%%% Hard Limit
    H = double(hardlim(tempH));
case {'tribas'}
    %%%%%%%%% Triangular basis function
    H = tribas(tempH);
case {'radbas'}
    %%%%%%%%% Radial basis function
    H = radbas(tempH);
    %%%%%%%%% More activation functions
end
```

- En el **ENT**, este bloque de código:

- a) Presenta 5 funciones de activación (**FA**)
- b) Evalúa los datos de ENT dados en **temH**
- c) Asigna la evaluación a la variable **H**

Tareas supervisadas usando ELM.

- Durante el entrenamiento, este bloque de código:

- Calcula los pesos de salida y el tiempo de cómputo
- Genera las **Etq** automáticas que emite la ELM
- Valora el desempeño de la ELM para Regresión

```
%OutputWeight=pinv(H') * T';
OutputWeight=inv(eye(size(H,1))/C+H * H') * H * T';
%OutputWeight=(eye(size(H,1))/C+H * H') \ H * T';
end_time_train=cputime;
TrainingTime=end_time_train-start_time_train

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Calculate the training accuracy
%%Cálculo de las etiquetas de Salida
Y=(H' * OutputWeight)';
if Elm_Type == REGRESSION
    TrainingAccuracy=sqrt(std(T - Y))
end
```


Tareas supervisadas usando ELM.

```

% Calculate the output of testing
start_time_test=cputime;
tempH_test=InputWeight*TV.P;
clear TV.P; % Release input of
ind=ones(1,NumberOfTestingData);
BiasMatrix=BiasofHiddenNeurons(1,ind);
tempH_test=tempH_test + BiasMatrix;
switch lower(ActivationFunction)
case {'sig','sigmoid'}
    % Sigmoid
    H_test = 1 ./ (1 + exp(-tempH_test));
case {'tan','tanh'}
    % Sine
    H_test = sin(tempH_test);
case {'hardlim'}
    % Hard Limit
    H_test = hardlim(tempH_test);
case {'tribas'}
    % Triangular basis function
    H_test = tribas(tempH_test);
case {'radbas'}
    % Radial basis function
    H_test = radbas(tempH_test);
    % More activation functions ca
end
TY=(H_test' * OutputWeight)';
end_time_test=cputime;
TestingTime=end_time_test-start_time_test
    
```

- Durante la PRUEBA, este bloque de código:

- Combina los datos de **PRU** y los pasa por la **FA**
- Genera las **Etq** automáticas que emite la ELM
- Calcula el tiempo de cómputo

Tareas supervisadas usando ELM.

```
%Cálculo del desempeño de la red durante la prueba o validación
if Elm_Type == REGRESSION
    TestingAccuracy=sqrt(std(TV.T - TY)) %se cambia por std %
end
%Tasas de acierto para entrenamiento y prueba
if Elm_Type == CLASSIFIER
    %%%%%%%%% Calculate training & testing classification accuracy
    MissClassificationRate_Training=0;
    MissClassificationRate_Testing=0;

    for i = 1 : size(T, 2)
        [X, label_index_expected]=max(T(:,i));
        [X, label_index_actual]=max(Y(:,i));
        if label_index_actual~=label_index_expected
            MissClassificationRate_Training=MissClassificationRate_Training+1;
        end
    end
    TrainingAccuracy=1-MissClassificationRate_Training/size(T,2)
    for i = 1 : size(TV.T, 2)
        [X, label_index_expected]=max(TV.T(:,i));
        [X, label_index_actual]=max(TY(:,i));
        if label_index_actual~=label_index_expected
            MissClassificationRate_Testing=MissClassificationRate_Testing+1;
        end
    end
    TestingAccuracy=1-MissClassificationRate_Testing/size(TV.T,2)
end
```

- Este bloque de código:

- Para regresión**, evalúa el desempeño de la ELM en la prueba
- Para clasificación**, evalúa el desempeño de la ELM tanto en el **ENT** como en la prueba